FOR DOMINO ADMINISTRATORS

Workshop Admin Setup

Do this once, before the workshop, with Domino admin rights. It prepares everything that lives in the Admin panel (the two compute environments and the hardware tiers) so participants only select these resources and never build or configure them. Participants then follow the workshop guide.

ENVIRONMENTS	HARDWARE TIERS	BUILD TIME	WHERE
3	**3 kinds**	**~25 min**	**Admin panel**
CPU, GPU, code assistant	Small, Medium, GPU	three image builds	Manage Resources

What participants will expect to find

RESOURCE	NAME	USED IN WORKSHOP STEP
Compute environment	`Fraud-Detection-Workshop`	1 to 8 (jobs, endpoint, app)
Compute environment	`Fraud-Detection-LLM`	9 (fine-tuning Job and GPU app)
Hardware tiers	Small, Medium, a small endpoint tier	2, 3, 6, 7
Hardware tier	A GPU tier, 24 GB or more	9
Compute environment (optional)	`Fraud-Detection-CodeAssist`	Any step, when using Cline or Codex in a VS Code workspace
Data plane and GPU tier (optional)	Oracle Cloud GPUs	Hosting a model for the code assistants

1 Create the workshop environment

One CPU environment runs the data Jobs, the training Job, the model endpoint and the app. It needs Streamlit and a Python 3.12 compatibility fix that the standard environment

lacks.

- [ ] **Open the environments page.** In the Admin panel, go to Manage Resources, then Environments, and choose to create a new environment.

- [ ] **Name it and choose a base.** Name it exactly `Fraud-Detection-Workshop` and start from the Domino Standard Environment for Python 3.12.

  Participants select it by this name, and the guide refers to it by this name, so do not change the spelling.

- [ ] **Make it visible to participants.** Set its visibility so every workshop user can pick it (a global environment, or one shared with the workshop users or organization).

- [ ] **Paste the Dockerfile instructions** below into the Dockerfile instructions box. Leave the other settings at their defaults.

The last two blocks matter. Python 3.12 removed a function that Domino's model-serving layer still calls, and without the fix the model endpoint never starts. The model API build also needs the image to end on the non-root user.

DOCKERFILE INSTRUCTIONS

```
# Prerequisites for uWSGI
USER root
RUN apt-get update && apt-get install -y --no-install-recommends gcc && \
    apt-get clean && rm -rf /var/lib/apt/lists/*

ARG LDFLAGS=-fno-lto
ENV LDFLAGS=-fno-lto
ENV PYTHONPATH="${PYTHONPATH}:/mnt/code:/mnt"

# Packages used by the app and the EDA report
RUN pip install --no-cache-dir "setuptools<81" Flask Flask-Compress Flask-Cors
uwsgi six prometheus-client ydata_profiling streamlit st-pages streamlit-
extras "lxml[html_clean]"

# Python 3.12 removed configparser.SafeConfigParser; Domino's model-serving
harness still calls it
USER root
RUN printf '%s\n' \
  'import configparser' \
  'if not hasattr(configparser, "SafeConfigParser"):' \
  '    class SafeConfigParser(configparser.ConfigParser):' \
  '        def readfp(self, fp, filename=None):' \
  '            return self.read_file(fp, filename)' \
  '    configparser.SafeConfigParser = SafeConfigParser' \
  > /opt/conda/lib/python3.12/site-packages/sitecustomize.py

# Model API builds need the image to end on the non-root user
USER ubuntu
```

- [ ] **Build it and wait for it to finish.** Confirm the new revision shows as built and active.

If you edit the environment later, each save makes a new revision and the newest successful one becomes the active one. Older revisions are harmless history and do not need to be archived.

2 Create the GPU environment

A second environment adds the packages that serve and fine-tune the two open models. It builds on the first, so it must be built second.

- [ ] **Create a new environment named exactly `Fraud-Detection-LLM`.** Use `Fraud-Detection-Workshop` as its base environment, and give it the same visibility as in step 1.

- [ ] **Paste this into its Dockerfile instructions and build it.** The build downloads several GB, so allow about 10 minutes.

DOCKERFILE INSTRUCTIONS

```
USER root
RUN pip install --no-cache-dir vllm transformers peft accelerate openai
scikit-learn
USER ubuntu
```

- [ ] **Confirm the build succeeded** and the revision is active.

The model weights themselves are not in the image. They download on first use into the participant project's dataset, so the cluster needs outbound access to Hugging Face, or the models must be mirrored to an internal registry (see the provenance note in the workshop guide).

3 Check hardware tiers

Participants choose from the hardware tiers you provide. Make sure the tiers below exist and are available to the workshop users.

- [ ] **A Small and a Medium CPU tier.** In the Admin panel, under Manage Resources, open Hardware Tiers and confirm both exist. Small runs the data Jobs, Medium runs the training Job.

- [ ] **A small tier for model endpoints.** Confirm a tier is marked as available for model APIs, since the endpoint in step 6 uses the small one.

- [ ] **A GPU tier.** Confirm a GPU hardware tier exists and the cluster has a GPU node pool that matches it.

  Both open models run on one GPU. With 40 GB or more (for example an L40S 48 GB) the generator runs in full precision. A 24 GB card also works because the launcher loads the generator in 8-bit, at a small cost in quality. Smaller than 24 GB will not fit both models.

- [ ] **Make sure a GPU node is free or can scale up.** If the cluster has no free GPU node, participants' GPU runs sit in Pending until one appears.

- [ ] **Check who may use the GPU tier.** If tiers are restricted, give the workshop users access.

PLAN GPU CAPACITY BY HEADCOUNT

Each participant who reaches step 9 holds one GPU for the fine-tuning Job and again for the running app. A room of people at the same step can exhaust a small pool, so scale the node pool up beforehand, or let people take turns.

---

4 Code assistant environment optional

Build one environment that already contains the Cline extension (as a `.vsix` file), the Domino plugin for Cline and the tools it needs, so participants open a VS Code workspace and only paste in their own API key. Codex needs nothing extra: the Domino Standard Environment already includes the Codex extension, the Codex CLI and the Domino skills.

SOME OF THIS IS A BEST GUESS

The steps follow the AI coding assistants setup guide and the usual Domino and code-server behavior. File locations marked "confirm" can differ by Domino version, so expect to adjust them after the first test in step 6. Domino 6.3 or Domino Cloud is assumed for the pre-installed Codex tooling.

4a. Gather the files

- [ ] **Get the Cline `.vsix`.** The extension id is `saoudrizwan.claude-dev`. On a connected machine, download the version you approve from the Open VSX registry, which is the gallery code-server uses. Note the version number.

  If your cluster cannot reach the internet during an environment build, put the file on an internal artifact server or file share that the build can reach, and use that URL below. Scan it and record its checksum as you would any binary you bring in.

- [ ] **Get the Domino plugin for Cline.** The source is `github.com/dominodatalab/domino-cline-plugin`. If the build cannot reach GitHub, mirror the repository internally and use the mirror URL below.

- [ ] **Confirm the base has what the plugin needs.** It needs Python 3.11 or later (the workshop environment has 3.12), `git`, and `uv`, which the Dockerfile below installs.

4b. Create the environment

- [ ] **Create a new environment named `Fraud-Detection-CodeAssist`.** Use `Fraud-Detection-Workshop` as its base (it already has the workshop packages and the VS Code IDE), and give it the same visibility as in step 1.

  Participants then use this one environment for the whole workshop when they want an assistant. If you also want assistants in the GPU workspace, repeat this step with `Fraud-Detection-LLM` as the base.

- [ ] **Paste this into its Dockerfile instructions.** Replace the values in angle brackets. This bakes the extension and plugin into the image under `/opt/vendor` and pre-builds the plugin's Python dependencies so nothing needs the internet when a workspace starts.

DOCKERFILE INSTRUCTIONS

```
USER root

# 1. Cline extension as a .vsix file (swap in your internal URL if needed)
ARG CLINE_VERSION=<approved-version>
RUN mkdir -p /opt/vendor/vsix && \
    curl -fsSL -o /opt/vendor/vsix/cline.vsix \
      "https://open-vsx.org/api/saoudrizwan/claude-
dev/${CLINE_VERSION}/file/saoudrizwan.claude-dev-${CLINE_VERSION}.vsix"

# 2. Domino plugin for Cline (swap in your internal mirror if needed)
RUN git clone --depth 1 https://github.com/dominodatalab/domino-cline-
plugin.git /opt/vendor/domino-cline-plugin

# 3. uv, then pre-build the MCP server's Python environment
RUN pip install --no-cache-dir uv && chown -R ubuntu:ubuntu /opt/vendor
USER ubuntu
ENV UV_PYTHON_DOWNLOADS=never
RUN cd /opt/vendor/domino-cline-plugin/mcp-servers/domino_mcp_server && uv
sync
```

- [ ] **Paste this into the environment's Pre Run Script.** It runs when each workspace starts. It installs the extension from the baked-in file and links the Domino skills, workflows, rule and hook into the places Cline reads. It pre-fills the MCP server entry only if the user has none yet.

The links go in the user's home folder, which is kept across restarts only when Home Directory Persistence is on. Running this script on every start rebuilds them either way, so participants do not repeat the plugin setup.

PRE RUN SCRIPT

```
#!/bin/bash
P=/opt/vendor/domino-cline-plugin

# Cline extension, from the .vsix baked into the image (no internet needed)
code-server --install-extension /opt/vendor/vsix/cline.vsix \
  --extensions-dir ${HOME}/.local/share/code-server/extensions

# Domino skills, workflows, global rule and hook
mkdir -p ~/.cline/skills ~/Documents/Cline/Workflows ~/Documents/Cline/Rules
~/Documents/Cline/Hooks
for d in $P/skills/*/; do ln -sfn "$d" ~/.cline/skills/"$(basename "$d")";
done
for f in $P/workflows/*.md; do ln -sfn "$f"
~/Documents/Cline/Workflows/"$(basename "$f")"; done
ln -sfn $P/rules/domino.md ~/Documents/Cline/Rules/domino.md
ln -sfn $P/hooks/PostToolUse ~/Documents/Cline/Hooks/PostToolUse

# MCP server entry for Cline, only if the user has none (confirm the path on
first test)
S=~/.local/share/code-server/User/globalStorage/saoudrizwan.claude-
dev/settings
if [ ! -f $S/cline_mcp_settings.json ]; then
  mkdir -p $S
  cat > $S/cline_mcp_settings.json <<EOF
{"mcpServers":{"domino_server":{"command":"uv",
 "args":["--directory","$P/mcp-
servers/domino_mcp_server","run","domino_mcp_server.py"],
 "env":{"UV_OFFLINE":"1","UV_PYTHON_DOWNLOADS":"never"}}}}
EOF
fi
```

- [ ] **Build it** and confirm the revision is active.

4c. Domino settings and network

- [ ] **Turn on Home Directory Persistence** for workspaces, if your deployment offers it as an admin setting. Cline's saved API settings live in the home folder and are lost on restart without it.

  The setting name and location depend on your Domino version, so look for it in the admin configuration. Codex does not need it, because Domino already saves `~/.codex`.

- [ ] **Allow outbound access from workspaces to the model provider.** Cline and Codex call a model API at run time. Open the route to the provider's API (for example OpenAI) or to your internal OpenAI-compatible gateway, and to nothing else if you can.

  The Open VSX download and the GitHub clone are needed only at environment build time, not for participants.

- [ ] **Decide where API keys come from.** Each participant brings their own key, or you issue keys from a gateway. Never put a key in the environment or the Dockerfile. Users can set it as a Domino user environment variable or enter it in the extension.

- [ ] **Optional: preset Codex for an internal gateway.** If everyone uses the same gateway, add this to the Pre Run Script so a first `~/.codex/config.toml` exists. Replace the values, and confirm the gateway supports the Responses API.

  ADD TO THE PRE RUN SCRIPT

```
mkdir -p ~/.codex
if [ ! -f ~/.codex/config.toml ]; then
cat > ~/.codex/config.toml <<'EOF'
model_provider = "internal_gateway"
model = "<your-model-id>"
[model_providers.internal_gateway]
name = "Internal OpenAI-compatible gateway"
base_url = "https://gateway.example.internal/v1"
env_key = "GATEWAY_API_KEY"
wire_api = "responses"
EOF
fi
```

5 Oracle GPUs in a separate data plane optional

If the code assistants should use a model that runs on your own GPUs in Oracle Cloud, decide whether those GPUs are managed by Domino. If they are, they need a data plane and a hardware tier. If they are not, they need only a network path.

WE DO NOT KNOW YOUR ENVIRONMENT YET

This step is written from assumptions and is meant to be refined with you as you go. Where a name or setting is a guess it is marked, and the questions at the end of the step are the ones to answer first. Send what you find back and we will fix the steps.

5a. Choose the pattern

PATTERN	WHAT IT MEANS	HARDWARE TIER NEEDED?
A. Domino runs the model	Domino is installed or attached in the Oracle data plane. A model server such as vLLM runs on those GPUs as a Domino app, endpoint or Job, and the assistants call its URL.	**Yes.** A data plane, a GPU node pool and a hardware tier in that data plane.
B. The model is hosted outside Domino	Your team runs an inference service on the Oracle GPUs (for example on OKE or a GPU VM) and Domino only calls it over the network.	**No.** Workspaces run on ordinary tiers. You need a reachable URL, a key and an egress rule.

The workspace that hosts VS Code does not need a GPU either way. The assistant is a client that sends requests to a model, so the GPU is needed only where the model itself runs.

5b. Pattern A: Domino-managed GPUs in the Oracle data plane

- [ ] **Confirm the Oracle data plane is registered** with your Domino control plane and shows as healthy. If it is not, that is a separate, larger task to do first, with its own network, storage and identity requirements.

Guess: this is done under the admin settings for hybrid or remote data planes. Names vary by Domino version.

- [ ] **Confirm the GPU nodes are in a node pool** in that data plane, with the GPU drivers and the Kubernetes labels Domino uses to match hardware tiers to nodes.

- [ ] **Create a hardware tier on that data plane.** In Admin, under Manage Resources, then Hardware Tiers, create a tier, choose the Oracle data plane, point it at the GPU node pool, and set its CPU, memory and GPU count to fit the model.

  Tiers are grouped by data plane in the picker, so participants will see it under the Oracle data plane's name, not under Local. Give it a name that says what it is, such as `oci-gpu-a10-24gb`, and set which users and organizations may use it.

- [ ] **Make sure the environment works on that data plane.** The environment image must be pullable from the Oracle data plane, so check that its registry is reachable and the image has synced or can be pulled.

- [ ] **Serve the model on the GPU tier.** Run an OpenAI-compatible server such as vLLM there, using the Domino approach you prefer (a long-running app or a model endpoint), and record its URL.

  Cline works with any OpenAI-compatible endpoint. Codex expects the Responses API, so confirm your server version supports it, or use Cline for this model.

- [ ] **Check data access.** A project's files and datasets can live on one data plane while the model runs on another. Confirm what the Oracle data plane can read, since the workshop's fine-tuning Job (step 9) reads and writes the project dataset.

5c. Pattern B: a model hosted outside Domino

- [ ] **Get the model URL and a key scheme** from the team that runs the inference service on the Oracle GPUs, and confirm it speaks the OpenAI API.

- [ ] **Open the network path from Domino workspaces** to that URL. This is usually a firewall or security-list rule on the Oracle side and an egress rule on the Domino cluster, plus any private link or VPN between them.

5d. Either pattern: point the assistants at it

- [ ] **Give participants the base URL and model id.** In Cline they choose **OpenAI Compatible** and enter the base URL, key and model id. For Codex, use the gateway settings in step 4c.

- [ ] **Test one request from a workspace** with the command below. A JSON reply with the model id means the path works.

```
curl -s -H "Authorization: Bearer $GATEWAY_API_KEY"
https://gateway.example.internal/v1/models
```

QUESTIONS TO ANSWER FIRST

1. Is Domino running in Oracle, or are the GPUs managed by someone else (pattern A or B)? 2. Which Domino version, and is the Oracle data plane already registered? 3. What GPU model and count, and is it shared with other work? 4. Is the Oracle environment air-gapped from the internet, and from the primary data plane? 5. Which model will run, and does its server support the Responses API? 6. Where do project datasets live relative to the Oracle data plane? 7. Who issues API keys?

---

6 Test and hand off

Prove the environments work from a normal project before participants arrive, then tell them what to expect.

- [ ] **Do a dry run in a throwaway project.** Follow step 1 of the workshop guide as a participant would (both the Git and the Domino-hosted option), selecting `Fraud-Detection-Workshop` in project Settings.

- [ ] **Run the environment check** from a workspace terminal. Every line under "Required" must say `ok`.

```
python .setup/verify_environment.py
```

- [ ] **Start a workspace on a GPU tier** with `Fraud-Detection-LLM` and confirm it reaches Running.

```
nvidia-smi
```

- [ ] **If you built the code assistant environment (step 4), test it.** Start a VS Code workspace with `Fraud-Detection-CodeAssist`. Confirm the Cline icon is in the sidebar, `ls ~/.cline/skills` lists the Domino skills, and `domino_server` shows connected under Cline's MCP Servers. Then ask Cline to list your Domino clusters.

If the extension or skills are missing, read the workspace's startup log for the Pre Run Script, then check the file locations marked "confirm" in step 4.

- [ ] **If participants will use the Git option, check Git access.** The workshop code is on GitHub at `github.com/DDL-Mike-Snyder/fraud-workshop`, and participants fork it. Confirm the Domino control plane can reach `github.com`, that users may create Git-based projects, and that a user can add a GitHub credential in Account Settings. In your dry run, create a Git-based project from a fork, unzip the code zip from `user/downloads/`, and run `.setup/unzip_dataset.ipynb`.

If GitHub is blocked, participants use the Domino-hosted option with the zip files instead. You can also copy the repository to an internal Git server and give participants that URL.

- [ ] **Send participants what they need:** the workshop guide, `fraud-workshop-code.zip` and `fraud-workshop-data.zip` from the `user/downloads/` folder, and the names of the hardware tiers to choose.

PARTICIPANTS NO LONGER BUILD ANYTHING

With the environments and tiers in place, participants only create a project, add the code and data, select an environment and run. If one cannot see an environment in the list, check its visibility (step 1, 2 or 4 above) first.